

告别 GPU 资源浪费：蚂蚁混部场景下的 GPU 虚拟化新范式

方睿 garry

蚂蚁 DeepXPU 团队 - 异构虚拟化与混部负责人



Content 目录

01 问题背景

02 现有方案局限性

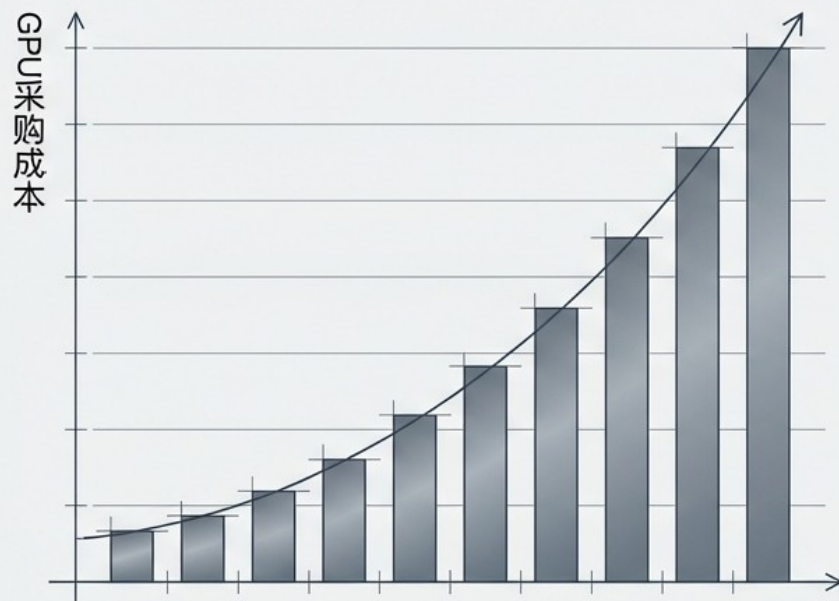
03 xGPU 解决方案

04 总结与展望

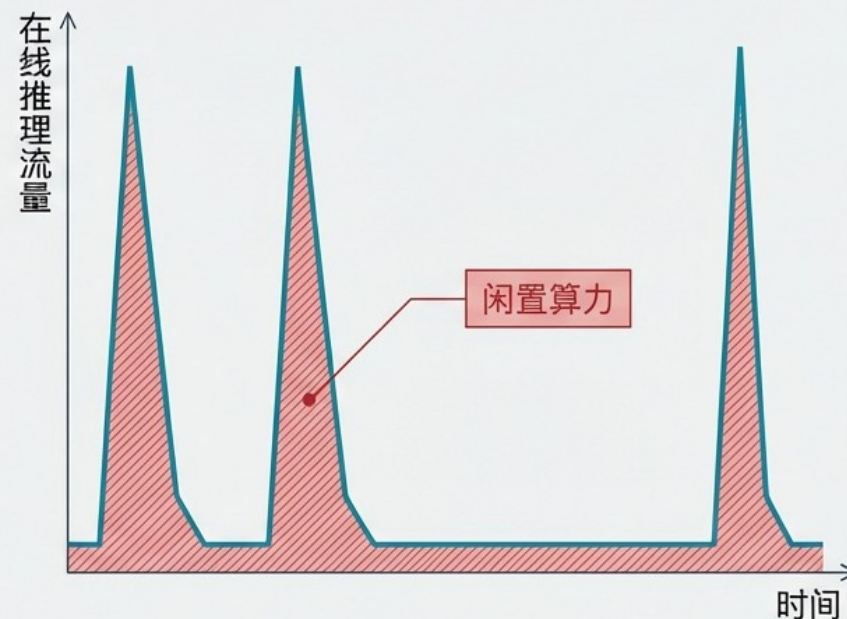


问题背景

算力洪荒时代的昂贵悖论



GPU 已成为数据中心最昂贵的核心硬件资源。



在线推理服务存在明显的“峰谷效应”与长尾特征。

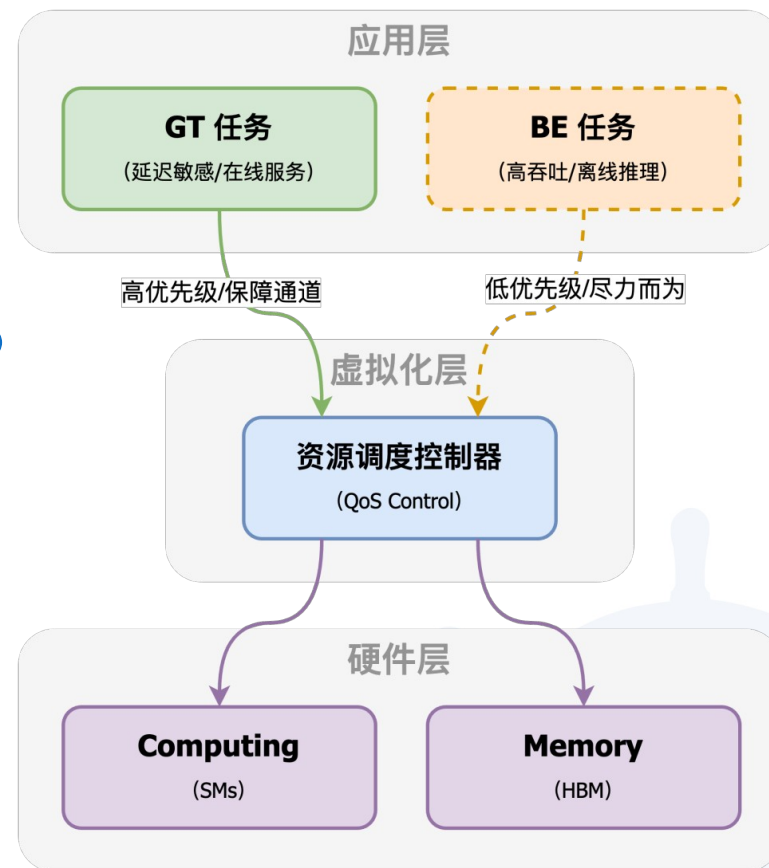
按照流量峰值预留资源 = 日常海量 GPU 算力闲置与极低的整体利用率。

推理任务分级

- GPU 推理服务按照 QoS 主要可以分为以下两大类：

- GT (Guaranteed) 任务**：SLA 要求较高的任务，一般对应在线服务，通常对请求压力 (**QPS**) 或访问延迟 (**RT**) 等指标有明确的要求，对资源质量较为敏感。GPU 集群资源的采购和预算一般会按照 GT 服务的流量峰值进行预留，如传统搜广推模型服务，GPU 利用率普遍偏低。
- BE (Best Effort) 任务**：SLA 要求较低的任务，一般对应离线任务，通常是一些计算密集型的高吞吐型任务类应用，有较好的容错重试能力，对资源质量的要求相对较为宽松，如离线数据清洗与处理等任务，GPU 利用率普遍较高。

- GT 的 SLA 保障
- 显存共享
- 故障隔离



Content 目录

01 问题背景

02 现有方案局限性

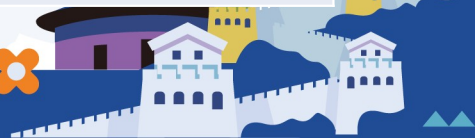
03 xGPU 解决方案

04 总结与展望



现有方案局限性

虚拟化方案	实现层级与技术原理	主要优势	主要局限性（在离线混部场景）
NVIDIA MIG	硬件与驱动层 通过静态硬切分实现	隔离性最强，能严格保障实例的 QoS（服务质量）。	1. 对 GPU 卡型与硬件架构有依赖； 2. 静态切分导致灵活性差、资源利用率低； 3. 存在 SM（流多处理器）切分损耗； 4. 不适合在离线混部场景。
NVIDIA MPS	软件层（软隔离） 通过合并 CUDA Context 实现（CUDA 13.1+ 支持 SM 数量配置）	支持软隔离与一定程度的计算资源（SM）配置。	1. 缺乏故障隔离，一个进程异常可能会导致其他进程崩溃； 2. 稳定性风险高，难以在生产环境中大规模推广。
基于 CUDA 拦截 （用户态软件方案）	用户态 通过拦截 CUDA API 来控制 Kernel 的下发速度	能够在切分场景下对任务下发进行一定的控制。	1. 无法控制已经 launch 的 Kernel，导致高优（GT）服务的 SLA 易受严重影响； 2. 侧重于单纯的资源切分而非多 QoS 等级混部，在大模型时代适用性受限。
基于 ioctl 拦截 （内核态软件方案）	内核态 拦截 CUDA 与设备底层的 ioctl 交互协议	能实现部分 CUDA 用户层无法做到的底层设备控制。	1. 处于设备控制层，缺乏上层应用信息； 2. 难以捕捉 CUDA 应用行为进行高视角的精细调度； 3. 无法独立完成复杂调度，需配合其他方案联合使用。



Content 目录

01 问题背景

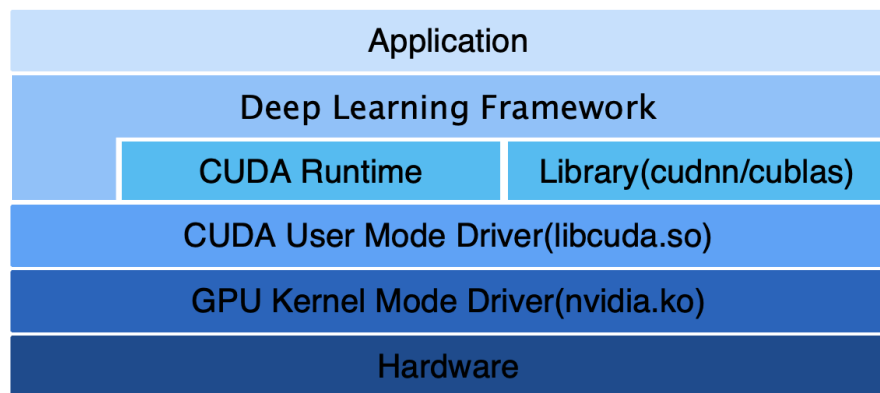
02 现有方案局限性

03 xGPU 解决方案

04 总结与展望



基本思路



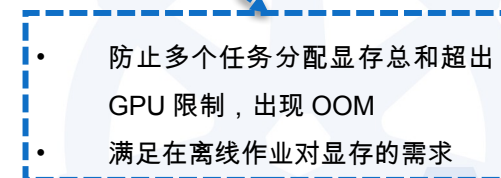
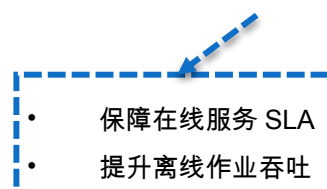
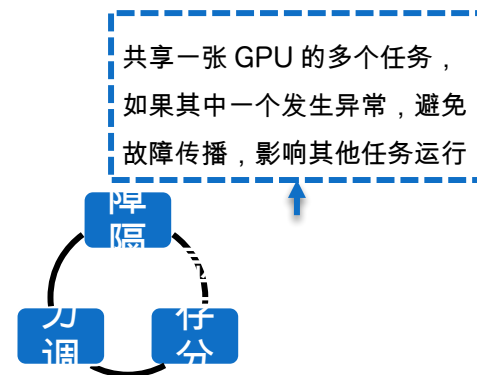
Deep Learning on GPU stack

Deep Learning Framework : ? 需要修改框架，优点是可以获取高层语义信息

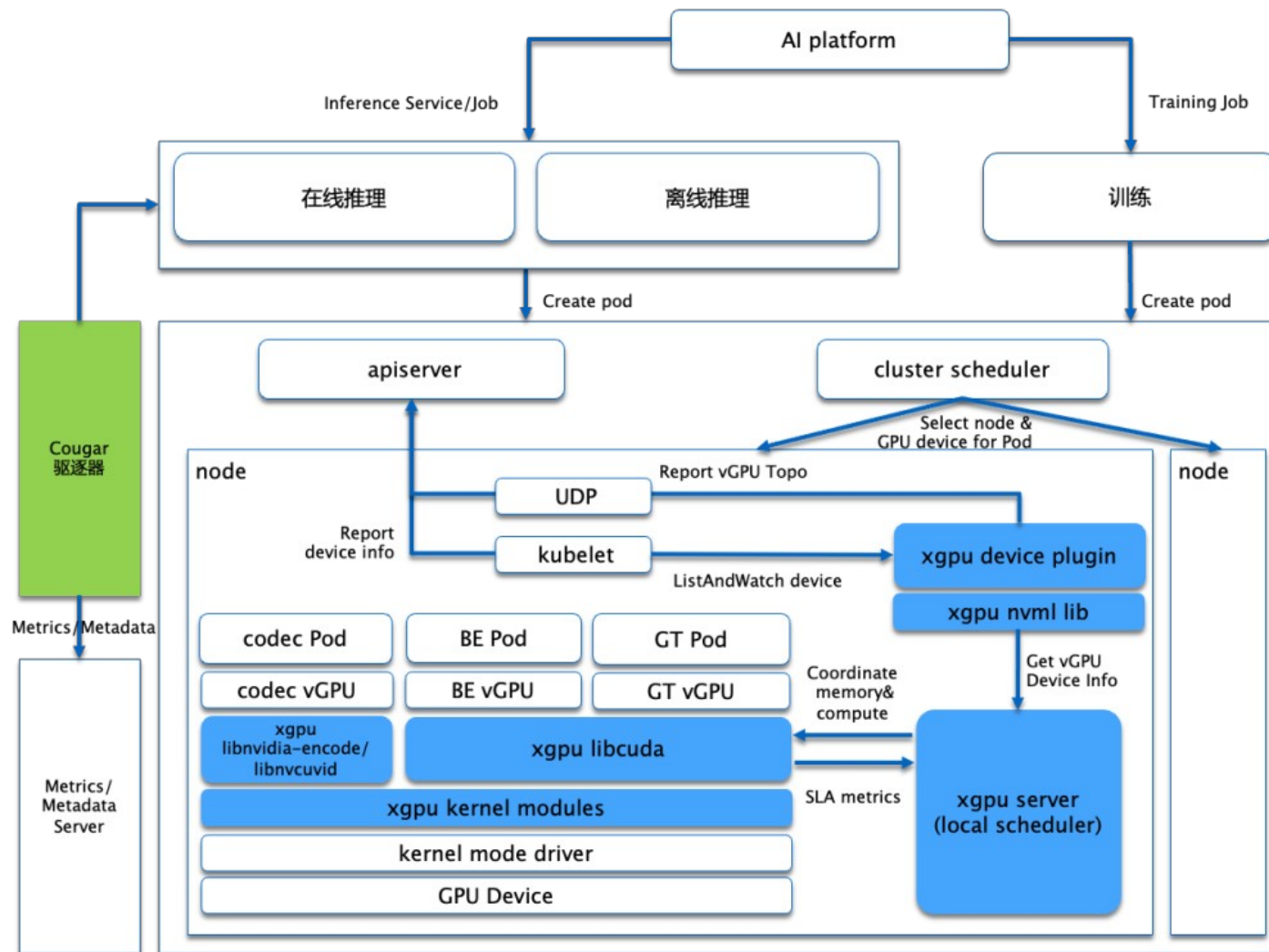
Runtime : ✗ Library 众多，且无法完整覆盖所有的调用

User Mode Driver : ✓ API symbol 公开，是 GPU 调用的入口

Kernel mode Driver : ✓ 现已公开，包含更精细化的控制能力

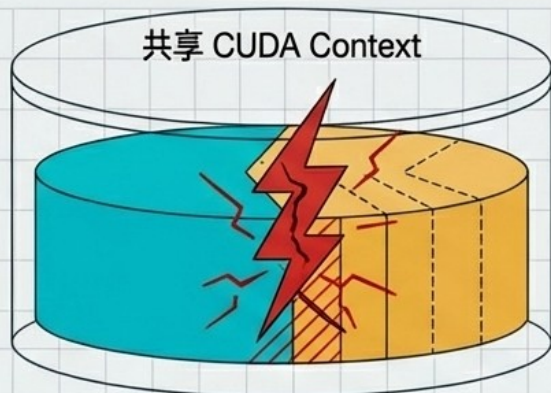


xGPU V3 架构

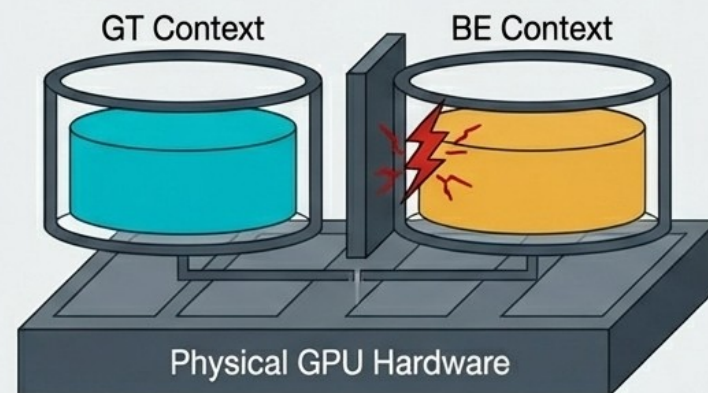


核心支柱一：基于独立 CUDA Context 的绝对故障隔离

Context 解析： CUDA Context 是绑定设备指针、流、核函数缓存等资源的核心容器。



MPS 的致命缺陷： 强制合并不同进程的 Context，一旦发生异常，故障直接跨进程传播。

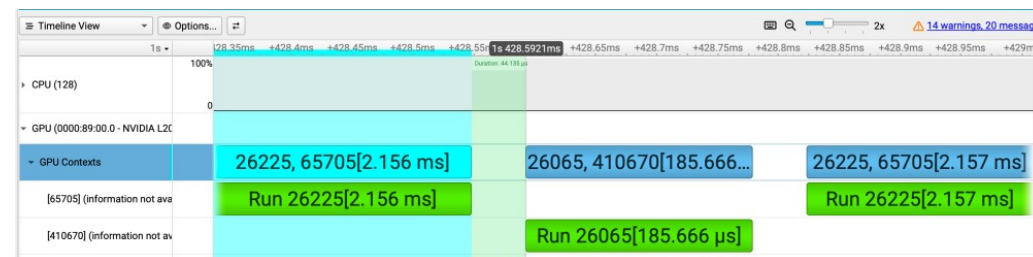


xGPU 隔离策略： 坚持多进程独立拥有自己的 CUDA Context。从物理层级切断故障传播链条，确保在线服务极度稳定。

算力控制（内核态）

- 多进程混部共享 GPU 时具有以下特征：
 - 当 Stream 上有未执行的 CUDA Kernel 时会被 GPU 硬件调度，运行期间的最长时间片为 **2ms**。
 - 当 Context 时间片耗尽后，若其他 Context 上的 Stream 中有待执行的 CUDA Kernel，会强制进行 Context 切换，切换时间在 **100 ~ 200us** 量级，具体会根据 GPU 硬件与 Kernel 执行情况决定。
 - 若 Context 上没有待执行的 CUDA Kernel，则 GPU 硬件调度器**不会为该 Context 对应的 TSG 分配时间片**，此时另一个进程在时间片耗尽后也不会进行 Context 切换。

- 内核模块劫持并转发 ioctl 请求
- 在内核层极度压缩 BE 任务的 Context 时间片



算力控制（用户态）

目标

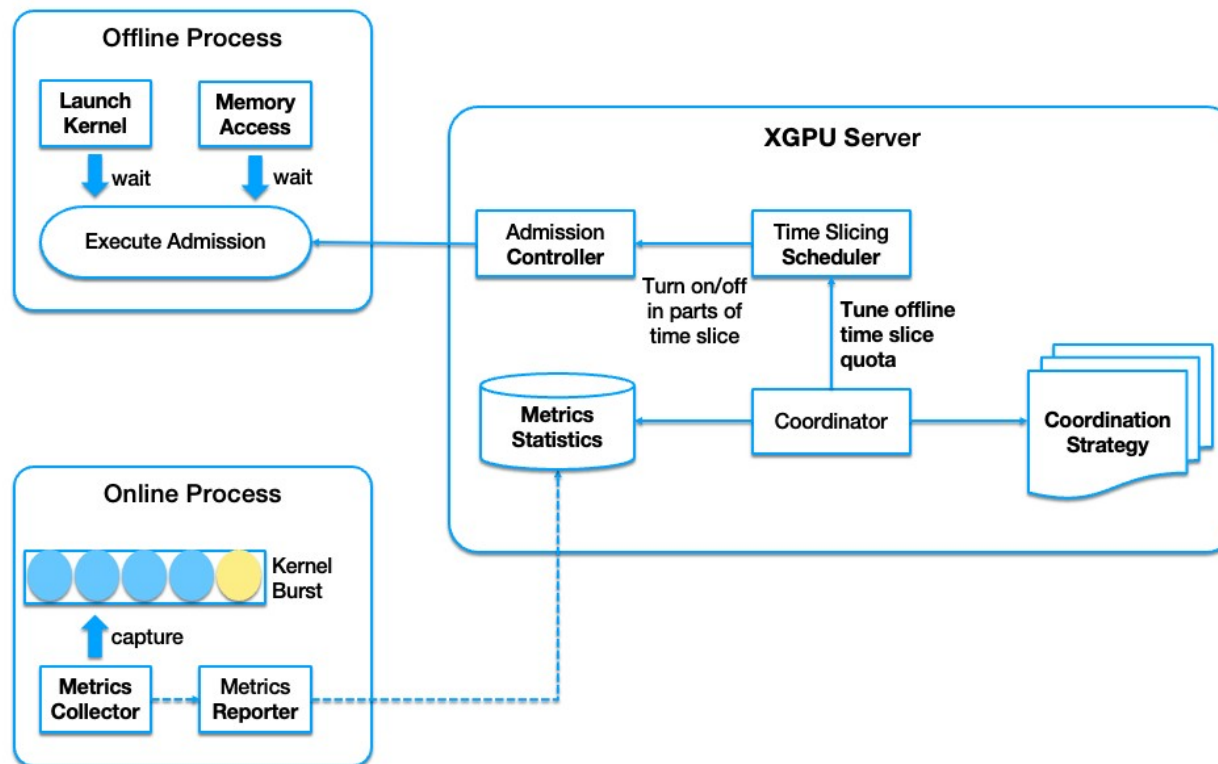
- P0: 保障在线推理服务SLA
- P1: 增加离线作业吞吐，提升利用率

作业优先级

- 高优: 在线推理，可全速 launch kernel
- 低优: 离线训练/推理，受时间片调度控制

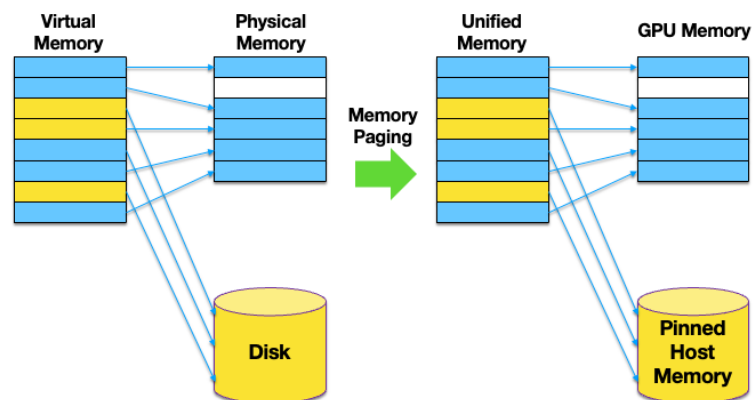
Metrics反馈调度

- 在线推理服务定时上报SLA Metrics
- xgpu server根据metrics反馈实时动态调节低优时间片，保障 SLA 的同时提升利用率



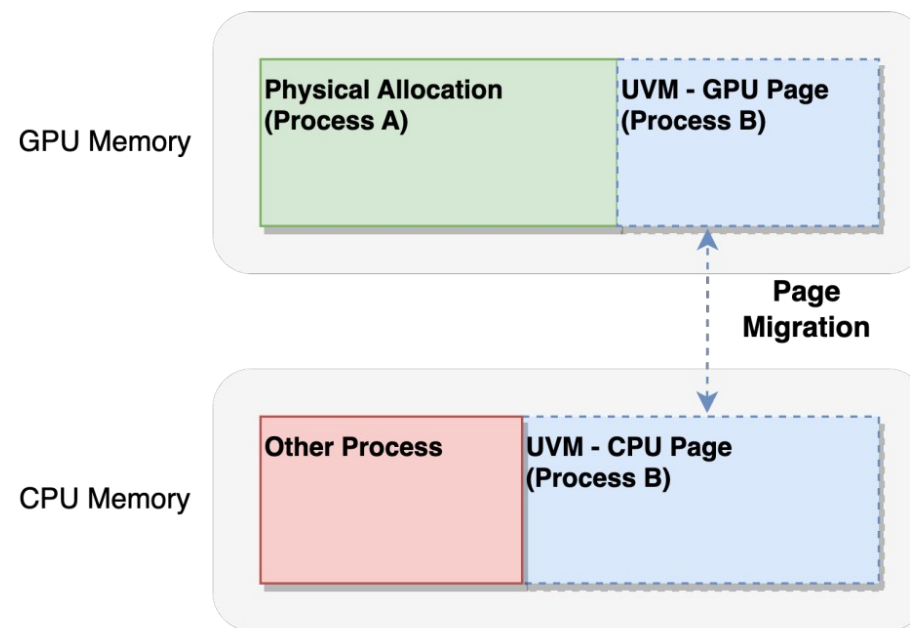
显存控制

显存超分



CUDA Unified Memory

- 离线显存使用超出上限，将memory chunk从物理显存swap到系统内存
- 弹性伸缩



Content 目录

01 问题背景

02 现有方案局限性

03 xGPU 解决方案

04 总结与展望



迈向大模型时代的算网融合

xGPU V3 已彻底解决传统推理场景下的 QoS 混部难题，告别算力浪费。

结合推理引擎底座，探究大模型 (LLM) 场景下的：

- 多租户大容量显存超分
- 多实例、多模型的毫秒级快速切换与混部

持续进化，以极致的底层调度，
释放每一 TFLOPS 的物理潜能。

NotebookLM

Thanks.

